In the following command cell, you customize the `mlflow.autolog` and start the mlflow experiment for training, testing, wrapping, and logging the model. You will use `mlflow.evaluate` to handle the evaluation. The logging process is easy. You call `log_model` with the model name, wrapped model, `pyfunc` flavor, and previously created training set. This process also registers the model in UC. Last in this notebook is a quick test on the predict function showing how to pass in the Spark context with the input data. You are now ready to validate the model.

Next, focus on the `CH7-07-Model Validation` notebook, which checks to ensure the input model has the correct metadata to be ready for production. This notebook can be used to test any model. Ideally, you will add numerous checks, including the ability to predict and possibly the testing accuracy on specific slices of data. For example, you could check the model performance on each product or geography. You can pass those columns with tags when slices need testing.

Collect the model details as shown in *Figure 7.18*. Notice the use of the util function, which is imported in a previous cell: `from mlia_utils.mlflow_funcs` import `get_latest_model_version`.

```
model_details = mlfclient.get_registered_model(model_name)
model_version = str(get_latest_model_version(model_name))
model_version_details = mlfclient.get_model_version(name=model_name, version=model_version)
```

Figure 7.18 – Use MLflow Client and util functions to access the model details

Each time you train and log the model, a new model version is created. Using tags, you can indicate which model version(s) must be tested and validated before deployment.

```
assert 'validation_status' in model_version_details.tags.keys(), f"the model, model={model_name},
specfied does not have validation_status tag"
if model_version_details.tags['validation_status'] == 'passed_tests':
  dbutils.notebook.exit("No validation needed!")

assert model_version_details.tags['validation_status'] == 'needs_tested', f"the latest version,
version={model_version}, of model, model={model_name} is not tagged as validation_status=needs_tested"
```

Figure 7.19 – Assertions to be sure the model needs to be tested

As shown in *Figure 7.20*, you need an informative model description for all production models. We recommend you include information about the use case the model is used for. Metadata hygiene is becoming increasingly important as companies want to leverage Generative AI on internal data, as LLMs use the metadata fields to find relevant information in the models.